BUILDER · PROBLEM SOLVER

만든 것을 사용자 앞에 내놓고, 반응을 보고 다시 고치는 빌더

문제 정의 · 빠른 출시 · 피드백 기반 개선 · 지표 설계

뷰티샵 원장님에게 직접 들은 불편에서 시작해 예약·고객관리 서비스 Vesta를 혼자 만들어 운영하고 있습니다. 기획·디자인·개발·배포·마케팅·CS를 모두 맡고 있고, Claude Code로 iOS·Android·Web 3플랫폼을 출시했습니다. 가입자 87명, 주간 활성 21명이라는 작은 규모지만 누가 왜 돌아오고 왜 떠나는지를 매일 확인하면서 기능을 고쳐 왔습니다.

김준영 | tsi04037@gmail.com | 010-4517-8157 | 경기 성남 분당
Vesta App Store · Google Play · vestaweb.net | 웹 포트폴리오 tei-young.github.io/builder

01 ABOUT

3년 7개월 동안 QA Engineer로 일하면서, 기능이 명세대로 동작하는 것과 사용자가 실제로 그 기능을 쓰는 것은 다르다는 걸 계속 확인했습니다. 커머스 회사에서는 전체 인기 상품을 추천하자는 기획에, 평소 관심 있거나 자주 사는 카테고리의 상위 상품이 더 매력적일 것이라는 가설로 '자주 구매하는 카테고리의 인기 상품'을 제안했습니다. A/B 테스트에서 클릭률이 10% 미만에서 18% 이상으로 올라 제안안으로 출시됐고, 같은 상품이라도 나에게 맞춰져 있을 때 더 많이 눌린다는 걸 그때 숫자로 확인했습니다.

Vesta는 회사에 다니면서 이미 만들기 시작한 서비스입니다. 지인 원장님에게 "기존 앱은 너무 어렵다, 간단하게 입력만 해서 예약·매출·고객을 다 관리하고 싶다"는 말을 듣고, 이렇게 들어가면 큰 시장은 아니어도 정말 필요한 원장님들에게는 닿을 수 있겠다고 생각했습니다. 사이드 프로젝트로 시작해 개인사업자로 전환했고, 만들면서 제 설계 의도와 사용자의 실제 행동이 다를 때 어느 쪽을 따라야 하는지를 가장 많이 고민했습니다. 어떤 피드백은 받아들여 제 방향을 바꿨고, 어떤 피드백은 서비스의 정체성을 이유로 반영하지 않았습니다.

혼자 만드는 일은 재미있지만 무엇을 무료로 두고 어디서 돈을 받을지, 어느 채널에 시간을 쓸지 같은 사업적인 판단은 매번 작은 근거로 내려야 했고, 그 부분이 가장 부족하다고 느꼈습니다. 레브잇 PMF팀에서 하나의 카테고리를 사업처럼 처음부터 끝까지 맡으면서 그 판단을 더 많은 고객과 데이터 위에서 배우고 싶습니다.

대신 바로 보탤 수 있는 경험도 있습니다. 고객을 직접 만나 문제를 정의하고, 3~5일 주기로 기획부터 출시까지 혼자 돌려 본 경험은 가설을 빠르게 고객 앞에 내놓아야 하는 Problem Solver의 일과 같은 방식입니다. 커머스에서는 주문·결제·배송 흐름을 사용자 관점에서 검증했고 추천 기획에 개인화 대안을 내 A/B로 확인해 봤습니다. 또 AI가 만든 결과물을 판정 기준을 세워 검증한 뒤 실무에 써 왔기 때문에, 쇼핑 AI 에이전트가 내놓는 추천과 답이 고객에게 믿을 만한지 확인하는 일에서도 이 경험을 살릴 수 있다고 생각합니다.

문제를 현장에서 정의합니다

지인 원장님의 불편에서 시작했고, 출시 후에도 매장을 직접 찾아가 왜 쓰는지, 왜 안 쓰는지를 듣습니다.

가설을 빠르게 내놓습니다

기능 단위로 3~5일 주기로 기획부터 스토어 출시까지 반복하고, 큰 기능도 3주 안에 사용자 앞에 내놓습니다.

피드백으로 고칩니다

"템플릿 그대로 쓰고 싶다"는 말에 설계 방향을 바꿨고, "입력창이 안 보인다"는 말에 핵심 기능의 진입점을 바꿨습니다.

성공 기준을 먼저 정합니다

설치 수가 아니라 '첫 예약 등록률'을 주 지표로 두고, 기준선을 먼저 고정한 뒤 변화를 봅니다.

사용자를 직접 데려옵니다

스레드 네트워킹, 매장 방문, 지인 소개, 푸시 재유입까지 채널마다 어떤 사용자가 들어오는지 확인했습니다.

AI로 혼자 끝냅니다

Claude Code로 3플랫폼 출시와 운영 도구까지 만들고, 저는 기획·로직·UI/UX·비즈니스 판단에 집중합니다.

CASE 01

원장님들이 이미 쓰던 도구에서 가설을 찾고, 3주 만에 내놓아 확인하다

출시 푸시 1회로 휴면·기존 유저 10명 복귀

기존 링크 서비스와 예약의 단절을 가설로 · 디자이너·원장님 자문으로 완성도 확보 · 출시 직전 구독 구조와 템플릿 추가

가입자는 87명이 되었지만 주간 활성은 21명이었습니다. 설치만 해두고 쓰지 않는 원장님들을 다시 데려와야 했고, 새로운 원장님이 들어올 길도 필요했습니다. 그래서 새 기능을 고민하기 전에 원장님들이 지금 무엇을 쓰고 있는지부터 살펴봤습니다.

현장 관찰
개인 링크 · 카톡·인스타 → **가설**
링크와 예약을 연결 → **자문 · 클로즈 베타**
디자인 · 사용 경험 → **출시 · 측정**
푸시 · 팝업 퍼널

1 원장님들이 이미 쓰던 것에서 시작하기

- 원장님들은 링크트리, 인포크 같은 서비스로 개인 링크를 만들어 카카오톡과 인스타그램 프로필에 걸어두고, 예약 현황·공지사항·카카오톡 채널·인스타그램으로 손님을 보내고 있었습니다.
- 그런데 이 링크들은 '내 예약 서비스'와 거의 연결되어 있지 않았습니다. 네이버 예약처럼 손님이 예약을 요청하고 원장님이 수락·거절하는 흐름이 링크 안에서 이어지지 않았습니다.
- 따라서 예약 관리 서비스인 Vesta가 이 링크를 무료로 제공하고 예약 시스템과 연동하면 링크에서 예약까지 한 흐름으로 이어지고, 그 과정에서 유일한 BM인 알림톡 사용도 늘어날 것이라고 가설을 세웠습니다.

2 손님에게 보이는 화면이라 완성도를 먼저 챙기기

- 손님에게 그대로 보이는 페이지라 무엇보다 미적인 부분이 중요했습니다. 디자이너 지인과 실제 원장님들께 사용 경험과 디자인 방향을 계속 자문받다 보니 기획부터 출시까지 테스트를 포함해 3주가 걸렸고, 지금까지 만든 기능 중 가장 긴 기간이었습니다.
- 출시 직전에는 템플릿 방식(CASE 02)과 함께 구독 구조를 더했습니다. 이미지를 많이 쓰는 페이지라 구현한 코드 기준으로 서버비를 계산해 보니, 방문자가 늘수록 제 부담이 커지는 구조였습니다. 어느 정도 적자는 감수하고 운영해 왔지만 이 기능은 무료만으로는 유지하기 어렵다고 판단해, 이미지를 일정 개수 이상 쓰거나 컬러를 더 다양하게 고르려면 구독하도록 바꿨습니다.

3 출시하고, 누가 돌아오는지 확인하기

- 출시일에 앱을 설치한 전체 사용자에게 '베스타 링크'가 나왔다는 푸시를 한 번 보냈습니다. 이 푸시로 기존 사용자와 한 달 이상 접속하지 않았던 휴면 사용자 10명이 돌아와 베스타 링크를 만들었고, GA4 이벤트로 확인했습니다.
- 앱 진입 시 베스타 링크 제작을 안내하는 팝업도 넣었습니다. 이번 주 기준 노출 32회, 클릭 8회, 게시 4건으로, 이미 링크를 가진 사용자가 많아 팝업 전환이 낮게 나오는 것으로 보고 있습니다.
- 링크를 거친 예약과 알림톡 사용량은 이제 집계를 시작한 단계라, 가설의 나머지는 이 두 숫자로 확인할 예정입니다.

> 원장님들이 링크 서비스를 따로 쓰고 있다는 것 자체가 이 기능이 필요하다는 근거라고 봤습니다. 제가 새로 만든 건 그 링크와 예약을 잇는 부분이었습니다.

결과
- 출시 푸시 1회로 휴면·기존 유저 10명 복귀, 베스타 링크 생성
- 기획부터 출시까지 3주, 서버비를 근거로 무료 범위와 구독 범위를 출시 전에 나눔

CASE 02

내 설계 방향보다 사용자의 속도를 택하다

베스타 링크 빌드 진입 후 90% 이상 게시 완료

"템플릿 그대로 쓰고 싶다"는 피드백으로 선택 폭을 줄임 · "입력창이 안 보인다"는 피드백으로 핵심 기능 진입점 개편 · 덜 쓰게 하고 싶은 경로는 의도적으로 깊게

Vesta는 '하나하나 내 맞춤으로 설정해 두면, 나중에는 조금만 조작해도 다 관리되는 앱'을 목표로 만들었습니다. 그런데 사용자가 원하는 게 처음부터 많이 설정하는 것보다 지금 당장 빨리 끝내는 것일 때가 있었고, 그럴 때는 제 설계 방향보다 사용자의 속도를 따랐습니다.

피드백 수집
클로즈 베타 · 사용 관찰
→
원인 파악
선택이 많음 · 진입점이 안 보임
→
구조 변경
템플릿 · + 버튼 · 2depth
→
결과 확인
게시 완료율

1 선택지를 줄여서 빠르게 — 베스타 링크 템플릿

- 출시 전 일부 원장님들께 클로즈 베타처럼 베스타 링크 빌드를 먼저 써보게 했습니다. 저는 Vesta 전체의 방향대로 항목 하나하나를 직접 설정하도록 유도했는데, "대충 만들어져 있는 템플릿을 그대로 쓰고 싶다"는 피드백이 나왔습니다.
- 제 방향과는 달랐지만, 원장님들이 원한 건 링크를 빨리 만들어 거는 것이었습니다. 그래서 대표 템플릿 2종 중 하나를 고르고 색상·버튼·이미지만 바꾸는 방식으로 선택의 폭을 오히려 줄였습니다.
- 설정 시간이 크게 줄었고, 지금은 베스타 링크 빌드에 진입한 원장님의 90% 이상이 게시까지 완료합니다.

2 익숙한 자리로 옮기기 — 간편입력 진입점

- 간편입력은 "김민지 3시 눈썹문신"처럼 문장 한 줄로 예약을 만드는 Vesta의 핵심 기능입니다. 처음에는 Claude나 ChatGPT처럼 일별 시트 하단에 입력창을 항상 띄워 두었습니다.
- 그런데 입력창이 눈에 잘 안 띈다는 피드백이 많았습니다. 원장님들에게 익숙한 건 채팅 입력창보다 캘린더 앱의 + 버튼이었기 때문에, 여러 캘린더 서비스처럼 + 버튼을 누르면 입력창이 나오도록 바꿨습니다.

3 덜 쓰게 하고 싶은 경로는 깊게 — 직접입력 2depth

- 반대로 항목을 하나씩 고르는 직접입력은 + 버튼 다음에 [직접 선택해서 입력]을 한 번 더 눌러야 하는 2depth로 내렸습니다.
- 간편입력이 더 빠르고, 파싱에 실패해도 입력한 내용이 상세입력으로 넘어가기 때문에 손해가 없다는 걸 사용자가 자연스럽게 익히도록 의도한 구조입니다.

> 피드백을 받으면 서비스의 정체성을 바꾸는 요청인지, 같은 일을 더 빨리 끝내게 해주는 요청인지부터 나눠 봤습니다. 템플릿과 + 버튼은 뒤의 경우라서 제 설계를 바꿨고, 앞의 경우는 CASE 06처럼 반영하지 않았습니다.

결과
- 템플릿 방식 도입 후 베스타 링크 빌드 진입 대비 90% 이상 게시 완료
- 간편입력을 기본 경로로, 직접입력을 보조 경로로 재배치해 핵심 기능이 먼저 보이도록 개편

CASE 03

핵심 기능을 원장님이 실제로 일하는 방식에 맞춰 넓히다

간편입력: 단건 → 복수 예약 → 카카오톡 메시지 붙여넣기

문장 한 줄 예약에서 시작 · 실제 예약이 들어오는 모습에 맞춰 입력 형태 확장 · 실패해도 상세입력으로 이어지는 안전망

간편입력의 첫 버전은 "김민지 3시 눈썹문신" 같은 한 건의 예약만 이해했습니다. 그런데 원장님들의 예약은 한 건씩 깔끔하게 들어오지 않습니다. 여러 건을 한 번에 적어야 할 때도 있고, 이미 카카오톡으로 손님에게 보낸 예약 메시지가 있기도 합니다. 그래서 원장님이 이미 가지고 있는 텍스트를 그대로 넣을 수 있게 입력 형태를 넓혔습니다.

첫 버전	→	확장 1	→	확장 2	→	안전망
단건 예약		복수 예약 한 번에		카카오톡 예약완료 메시지		실패 시 상세입력으로

1 한 번에 여러 건

- "김민지 3시 눈썹문신 / 김하율 5시 눈썹문신"처럼 여러 예약을 한 번에 입력해도 각각의 예약으로 나눠 저장되도록 확장했습니다.

2 이미 가진 텍스트를 그대로

- 원장님들은 손님에게 카카오톡으로 예약 확정 메시지를 직접 보내거나 자동으로 발송합니다. 이 메시지를 복사해 그대로 붙여넣으면 그 안에서 이름·시간·시술 정보를 찾아 예약을 만드는 기능을 추가했습니다.
- 사용자 요청이 아니라 제 아이디어였습니다. 예약 정보가 이미 카카오톡에 텍스트로 남아 있는데 그걸 다시 입력하게 하는 건 불필요한 일이라고 생각했습니다.

3 실패해도 멈추지 않게

- 자유 입력은 언제든 해석에 실패할 수 있습니다. 실패하면 입력한 내용이 상세입력 화면에 채워진 채로 넘어가도록 해서, 간편입력을 시도하는 것 자체가 손해가 되지 않게 했습니다.
- CASE 02에서 직접입력을 2depth로 내릴 수 있었던 것도 이 안전망이 있었기 때문입니다.

결과

- 단건 → 복수 예약 → 카카오톡 메시지 붙여넣기로 입력 형태 확장
- 간편입력이 '예약 저장 하나로 고객 장부·매출이 채워지는' 메인 여정의 시작점이 됨

CASE 04

무엇을 성공으로 볼지 먼저 정하고, 숫자를 의심하다

주 지표 '첫 예약 등록률' · 24시간 내 45% (기준선 43%)

설치 수가 아닌 첫 예약 등록을 주 지표로 · 기준선을 먼저 고정 · 테스트 데이터로 오염된 진단은 폐기

가입자 수십 명 규모에서는 숫자 몇 개만 흔들려도 결론이 바뀝니다. 그래서 무엇을 성공으로 볼지 먼저 정했고, 그 숫자가 정말 사용자의 행동을 보여주는지부터 확인했습니다.

지표 정의	→	계측	→	정리	→	해석
첫 예약 등록률		GA4 3플랫폼 동일 명세		테스트 트래픽 제외 · 코호트		기준선 대비 변화

1 설치 수가 아니라 첫 예약 등록

- Vesta는 예약을 저장해야 고객 장부와 매출이 채워지는 구조입니다. 앱을 설치하거나 가입한 것만으로는 아무것도 시작되지 않기 때문에 주 지표를 '첫 예약 등록률'로 정했습니다.
- 지표를 처음 만든 시점까지의 전환율을 기준선(24시간 내 43%, 7일 내 59%)으로 고정해 두고, 이후 기능을 고치거나 추가할 때마다 이 기준선과 비교합니다.
- 같은 이유로 앱을 열기만 한 사용자와 예약을 완료한 사용자를 처음부터 다르게 구분하고, 실제로 예약을 등록하며 앱으로 샵을 운영하는 사용자만 활성으로 집계합니다. 이 기준으로 본 핵심 사용자는 6명에서 12명으로 늘었습니다.

2 숫자를 먼저 의심하기

- iOS·Android·Web 3플랫폼에 같은 이벤트 명세로 GA4를 심고, 일일 지표 리포트로 퍼널과 코호트를 매일 확인합니다.
- 그런데 제가 QA를 하면서 가입·탈퇴 테스트를 반복하다 보니, 지표에 실제 사용자가 아닌 신규 가입과 이탈이 계속 쌓이고 있었습니다. 이 데이터로 본 퍼널 진단은 폐기하고, 테스트 계정과 테스트 형태의 트래픽을 모두 제외한 뒤 다시 봤습니다.
- 코호트를 전수로 다시 분석한 결과 이탈은 대부분 가입 당일에 결정되고, 그 갈림길이 첫 예약 등록이라는 걸 확인했습니다. 지금 첫 예약 등록률을 주 지표로 두는 근거입니다.

3 기준선 대비로 해석하기

- 현재 24시간 내 첫 예약 등록률은 45%로 기준선보다 조금 높고, 7일 내는 52%로 기준선보다 낮습니다.
- 최근 마케팅 대상을 '예비 창업 원장님'까지 넓히면서, 아직 손님이 없어 앱을 열어만 보거나 가입만 하는 원장님 비율이 늘어난 영향으로 보고 있습니다. 그래서 숫자가 내려갔을 때 바로 기능 문제로 보지 않고, 그 기간에 들어온 사용자가 달라졌는지부터 확인합니다.

결과

- 주 지표와 기준선을 먼저 고정해, 기능을 바꿀 때마다 같은 기준으로 비교
- 테스트 데이터로 오염된 진단을 폐기하고 테스트 계정·트래픽 제외, 이탈의 갈림길을 첫 예약 등록으로 확인

CASE 05

사용자를 어디서 데려오는지, 어떤 사용자가 남는지 확인하다

스레드 활동일 일 가입 0~2명 → 4~6명 · 매장 방문 2곳 모두 전환

채널마다 들어오는 사용자의 성격이 다름 · 가입 수와 활성 전환을 나눠서 봄 · 앱 안에서는 푸시와 팝업으로 재유입

처음에는 제가 직접 알리는 방법부터 시작했습니다. 채널을 늘리는 것보다, 채널마다 어떤 사용자가 들어오고 그중 누가 실제로 남는지를 나눠서 보는 데 집중했습니다.

직접 알리기	→	가입	→	활성 전환	→	재유입
스레드 · 방문 · 지인		일 가입 수		첫 예약 · 헤비유저		푸시 · 팝업

1 가입을 늘리는 채널 — 스레드

- 스레드에서 원장님들과 네트워킹하며 직접 Vesta를 알렸습니다. 평소 일 가입자는 0~2명인데, 스레드 활동을 많이 한 날에는 4~6명으로 확실히 늘었습니다.
- 다만 이 채널로 들어온 원장님 중에는 예비 창업 원장님도 많아서, 가입은 늘어도 바로 예약을 등록하지 않는 경우가 있었습니다 (CASE 04).

2 남는 사용자를 만드는 채널 — 매장 방문과 지인 소개

- 매장 두 곳을 직접 방문해 소개했고, 두 곳 모두 가입해 실제로 쓰는 헤비유저가 되었습니다. 그중 한 곳은 기존에 쓰던 다른 예약 관리 서비스에서 Vesta로 옮겨왔습니다.
- 지인 소개로 들어온 원장님들은 활성 사용자로 이어지는 경우가 많았습니다. 가입 수는 적어도 남는 비율이 높은 채널이었습니다.

3 이미 설치한 사용자를 다시 데려오기

- 새 기능이 나오면 FCM으로 전체 사용자에게 알리고, 앱 안에서는 팝업으로 다음 행동을 안내합니다. 베스타 링크 출시 푸시 1회로 휴면·기존 사용자 10명이 돌아왔습니다(CASE 01).
- GEO·SEO는 설정만 해둔 상태라 아직 효과를 측정하지 않았고, 지면 광고는 전환보다 인지도를 위한 채널로 검토하고 있습니다.

> 스레드는 가입을, 매장 방문과 지인 소개는 오래 쓰는 사용자를 데려왔습니다. 그래서 채널을 가입 수 하나로 비교하지 않고 활성 전환까지 나눠서 봅니다.

결과
- 스레드 활동일 일 가입 0~2명 → 4~6명, 매장 방문 2곳 모두 헤비유저로 전환(1곳은 기존 서비스에서 이동)
- 가입자 87명, 주간 활성 21명·월간 활성 35명, 핵심 사용자 12명

CASE 06

무엇을 무료로 두고, 무엇을 거절할지 정하다

유료 기능 결제 0건 → '전 기능 무료 + 알림톡 크레딧'으로 BM 전환

원장님들이 기존 앱에 돈을 내기 싫어한 이유에서 BM을 다시 설계 · 서버비를 계산해 무료 범위를 정함 · 정체성에 맞지 않는 요청은 반영하지 않음

처음에는 일부 기능을 유료로 두었지만 결제는 0건이었습니다. 원장님들에게 들은 이야기를 다시 보니 가격 자체보다 '필요 없는 기능까지 묶어서 돈을 낸다'는 데 대한 거부감이 컸습니다.

사용자 목소리
구독료 부담 · 끼워팔기
→
BM 전환
전 기능 무료 + 알림톡 크레딧
→
비용 계산
월 적자 상한 · 서버비
→
정체성 유지
반영하지 않을 기준

1 돈을 내기 싫은 이유에서 다시 시작하기

- 기존 뷰티샵 관리 앱에 대해 원장님들께 가장 많이 들은 이야기는 "구독료가 조금만 들어도 부담스럽다", "진짜 필요한 기능에 돈을 내는 게 아니라 끼워팔기라서 내기 싫다", "UI와 디자인이 구리다"였습니다.
- 유료 기능 결제가 0건이고 수요가 알림톡에 몰려 있다는 걸 근거로, 모든 기능을 무료로 열고 실제로 쓰는 만큼만 내는 알림톡 크레딧 과금으로 바꿨습니다. 앞으로의 유료화도 '주요 기능은 무료, 추가 기능만 개별 유료'라는 원칙으로 갈 계획입니다.

2 적자의 상한을 먼저 계산하기

- 전 기능을 무료로 열기 전에 제가 매달 감당할 수 있는 적자 상한부터 계산했습니다. 현재 월 운영비는 1~2만 원 수준입니다.
- 베스타 링크처럼 사용량에 따라 비용이 커지는 기능은 서버비를 따로 계산해서, 기본 기능은 무료로 두고 이미지·컬러 확장만 구독으로 나눴습니다(CASE 01).
- 알림톡은 돈이 오가는 경로라서 발송 큐, 발송 직전 5단계 체크, 크레딧 원장을 먼저 구조로 설계했습니다. 중복 발송이나 차감 오류가 나지 않게 하는 게 우선이었고, 현재 알림톡 발송은 주 50건 내외입니다.

3 반영하지 않을 피드백 정하기

- Vesta는 Timeblocks처럼 캘린더를 중심으로 한 간단한 일정 관리 앱을 지향합니다.
- 다른 뷰티샵 관리 앱들처럼 첫 화면을 캘린더가 아닌 다른 화면으로 바꾸자는 요청, 결산 기능은 필요 없다는 의견도 있었지만 반영하지 않았습니다. 캘린더 중심이라는 Vesta만의 지향점은 그대로 두기로 했습니다.

결과
- 유료 기능 결제 0건을 근거로 BM 전환, 월 운영비 1~2만 원 수준에서 전 기능 무료 유지
- 비용이 커지는 기능은 출시 전에 무료·구독 범위를 나누고, 정체성에 맞지 않는 요청은 기준을 세워 반영하지 않음

CASE 07 애즈위메이크 · QA Engineer · 2025.09 ~ 2026.07 · 커머스 B2C(앱·웹)·B2B(POS 연동·상품관리)

커머스 사용자의 행동을 기준으로 기획과 정책을 바꾸다

상품 추천 클릭률 10% 미만 → 18% 이상 (2주 A/B 테스트)

추천 기획에 개인화 대안을 가설로 제안해 A/B로 검증 · 데이터는 정상이지만 사용자가 손해 보는 정책을 개선 · 마트 사장님 개통 현장의 불편을 개선 의견으로

마트 상품 비대면 배송 서비스(B2C)와 마트 사장님용 상품관리 솔루션(B2B)을 QA로 담당했습니다. 검증하는 역할이었지만, 사용자가 실제로 어떻게 사고 어떻게 다시 시도하는지를 기준으로 기획과 정책에도 의견을 냈습니다.

기획 확인
전체 인기 상품 추천
→
가설
자주 구매하는 카테고리
→
A/B 테스트
2주 · 클릭률
→
출시
제안안 채택

1 추천 기획에 대안을 가설로 내기

- 상품 추천 기능이 '전체 인기 상품'을 보여주는 방향으로 킥오프되었습니다. 전체 인기 상품도 충분히 매력적이지만, 저는 평소 관심 있거나 자주 사는 카테고리의 상위 상품이 더 매력적일 것이라고 생각했습니다. 나에게 맞춰진 상품일수록 더 많이 누르고 살 것이라는 개인화에 대한 가설이었습니다.
- 확신할 근거가 있는 건 아니었기 때문에 주장 대신 A/B 테스트를 제안했고, 클릭률 이벤트를 기준으로 2주간 비교했습니다. 기존안은 10% 미만, 제안안은 18% 이상으로 확인되어 제안안으로 출시되었습니다.

2 데이터는 정상이어도 사용자가 손해 보는 구조

- 알림 발송에 쓰이는 유상 재화가 외부 벤더 오류로 실패한 뒤에도 '소진' 상태로 남았습니다. 내부 배치가 주기적으로 복구하는 구조라 데이터 정합성 관점에서는 문제가 없었습니다.
- 하지만 실패한 사용자는 곧바로 재시도하고, 그때 복구돼 있지 않으면 잔액 부족으로 막히거나 이중 차감된 것처럼 보입니다. 이 구조를 근거와 함께 제기했고, 실패 시점에 즉시 복구되도록 정책이 바뀌었습니다.

3 현장의 불편을 개선 의견으로

- POS 연결과 상품 연동이 필요한 마트 사장님의 초기 개통·세팅 영역을 맡아, 개통 중 생기는 이슈를 팔로업하며 버그와 설정 문제를 구분했습니다. 반복되는 불편은 개선 의견으로 제안했습니다.

> QA를 하며 쌓은 '사용자가 다음에 무엇을 할까'에 대한 예상을 검증 범위를 잡는 데만 쓰지 않고, 기획 대안으로도 냈던 경험입니다.

결과
- 상품 추천 클릭률 10% 미만 → 18% 이상, 제안안으로 출시
- 사용자의 재시도 행동을 근거로 유상 재화 정책을 즉시 복구로 개선

HOW I BUILD

AI에게 구현을 맡기고, 판단과 검증을 맡다

3~5일 주기로 기획부터 스토어 출시까지 · 3플랫폼 1인 운영

Vesta의 코드는 Claude Code로 작성합니다. 저는 무엇을 왜 만들지, 로직과 정책이 맞는지, 화면이 원장님에게 자연스러운지, 비용과 BM에 맞는지를 판단하는 데 시간을 씁니다.

전략 세션
가설 · 시장 조사
→
기획 · 디자인 세션
예외 · 추가사항
→
개발 세션
최소 단위 기능 1개
→
QA
탐색 테스트 · 전체 풀런

1 세션을 역할별로 나누고, 기능은 최소 단위로

- 작업을 플랫폼별 개발 세션, QA 세션, 기획 세션, 디자인 세션, 비즈니스(전략) 세션으로 나눠 운영합니다.
- 비즈니스(전략) 세션에서 제 아이디어와 가설을 함께 검토하고 시장·웹 조사를 합니다. 여기서 정리한 판단을 기획·디자인 세션으로 옮겨 예외사항과 추가사항을 다듬은 뒤 개발 세션으로 넘깁니다.
- 개발 세션은 개발 → QA(UI, 실기기, 유저 플로우, 신규 기능, 영향받는 기존 기능) → 테스트로 이어지는 파이프라인으로 돌고, 한 사이클에는 가능한 한 최소 단위의 기능 하나만 만듭니다. 모든 개발이 끝나면 탐색적 테스트와 전체 QA 풀런을 돌린 뒤 출시합니다.

2 운영도 도구로

- 일일 지표 리포트 봇으로 퍼널·코호트를 매일 받아보고, 스토어 출시 감시 Slack 봇으로 심사·출시 상태를 확인합니다. 정부 지원사업 공고를 수집해 Vesta에 맞는지 판정하는 봇도 직접 만들어 운영합니다.
- 핵심 시나리오는 Maestro로 UI를 자동화하고, 웹은 배포 후 실서버 스모크 테스트로 동작을 확인합니다.

3 AI 결과물은 기준을 정해 검증하기 (애즈위메이크)

- QA 시절 Claude Code로 TC 생성 도구를 만들었을 때, 첫 결과물은 QA 엔지니어가 쓴 TC 기준 20~30%만 충족했습니다. 리뷰하면 직접 쓰는 것과 시간이 비슷하거나 더 드는 수준이었습니다.
- 'TC라면 지켜져야 하는 기준'을 먼저 판정 기준으로 고정하고, 반복되는 실패 유형을 역할별 서브에이전트로 나눠 충족률을 70~80%까지 올렸습니다. 자동화 코드는 앱에 버그를 주입해 의도한 이유로 실패하는지까지 확인했습니다.
- 그래서 Vesta에서도 AI가 만든 결과를 그대로 믿지 않고, 무엇을 기준으로 맞다고 볼지 먼저 정한 뒤 확인합니다. 개발 세션 안에 QA 단계를 따로 둔 것도 같은 이유입니다.

애즈위메이크

QA Engineer · 2025.09 ~ 2026.07 · 커머스 B2C(앱·웹)·B2B(POS 연동·상품관리)

1인 QA로 기획 정의부터 출시 후 모니터링까지, 근거를 들고 일정을 판단하다

- TF별 1인 QA로 기획 정의부터 출시 후 모니터링까지 단독으로 맡았고, 동시에 최대 4~5개 프로젝트를 검증했습니다.
- QA 일정을 감에 근거해 산정하다 보니 일정이 모자라거나 남는 일이 잦아, 사내 QA 플랫폼을 직접 설계·개발했습니다 (React·TypeScript·Supabase → NCP·Docker, 약 60일). 테스터별 TC 처리 속도를 쌓아 QA 기간을 산정하자 예상과 실제 오차가 약 0.5MD로 줄었고, 이를 근거로 3개월·3회로 나눠야 했던 대형 릴리즈를 1개월·1회로 압축했습니다.
- 사업부의 핫픽스 요청에는 정규 배포라는 추천안과 영업 전담 대응이라는 대안을 함께 가져가 합의했고, 핫픽스는 담당 TF 기준 월 0~1회로 관리했습니다.

슈프리마

QA Engineer · 2025.02 ~ 2025.08 · 물리보안 SW

새로운 도메인에 빠르게 적응해 결함 검출율 50% 향상

- 웹툰 플랫폼에서 물리보안 솔루션으로 도메인을 바꿔 신규 제품 개발에 Scrum 일원으로 합류했습니다. 기존 TestCase와 Confluence 자료를 먼저 익히고, 2주 단위 Sprint의 Planning부터 참여해 User Case 기반으로 시나리오를 다시 설계했습니다.
- 출근 시간대 인증 요청 집중처럼 실제 고객이 제품을 쓰는 맥락을 시나리오로 만들고, 네트워크 불안정·동시 접속 증가·장시간 사용 상황을 직접 구성해 사용자 불편을 미리 찾았습니다.

디케이테크인(카카오웹툰)

QA Engineer · 2023.01 ~ 2025.02 · 콘텐츠 플랫폼(웹·앱)

서비스 운영 이슈의 원인을 규명해 관련 유저 CS 40% 감소

- 첫 단독 QA로 클라우드 이전 검증을 맡았습니다. 여러 작품의 회차가 노출되지 않는 이슈를 화면 리포트로 끝내지 않고 S3 버킷 이미지 문제까지 특정했고, admin 권한·유저 데이터 마이그레이션 같은 정책 영역까지 TestCase에 반영해 관련 유저 CS가 40% 이상 줄었습니다.
- Playwright·Selenium으로 Admin 반복 업무를 자동화해 테스팅 시간을 30% 이상 줄였고, 절감된 시간으로 major 이상 신규 이슈를 10건 이상 추가로 찾았습니다.
- 신규 AI 추천 기능(Helix)의 API 명세 확인과 기획에 참여하고 테스트 방식을 문서화했습니다.

04 CAREER

경력 요약 · 스킬 · 자격

2026.04 ~ 현재 **Vesta** 창업 · 1인 기획·개발·운영 · 뷰티샵 예약·고객관리

원장님 현장 피드백으로 문제를 정의하고 iOS·Android·Web을 혼자 출시·운영. 가입자 87명, 주간 활성 21명, 핵심 사용자 12명.

2025.09 ~ 2026.07 **애즈위메이크** QA Engineer · 커머스 B2C·B2B

상품 추천 개인화 대안을 A/B로 검증해 출시. AI TC 생성 도구와 사내 QA 플랫폼을 직접 만들어 대형 릴리즈 67% 단축.

2025.02 ~ 2025.08 **슈프리마** QA Engineer · 물리보안 SW

새 도메인에 Scrum 일원으로 합류해 User Case 기반 시나리오 설계, 결함 검출율 50% 이상 향상.

2023.01 ~ 2025.02 **디케이테크인(카카오웹툰)** QA Engineer · 콘텐츠 플랫폼

서비스 운영 이슈를 규명해 관련 유저 CS 40% 감소. Admin 자동화로 테스팅 시간 30% 단축.

2021.09 ~ 2021.12 **한국지능정보사회진흥원(NIA)** 인턴 · 공공 데이터 품질

공공 데이터셋의 정확성·일관성 보장을 위한 데이터 처리(우수인턴 수상).

SKILLS

기획 · 분석

현장 인터뷰 기반 문제 정의, GA4 3플랫폼 이벤트 명세 설계, 퍼널·코호트 분석, A/B 테스트

고객 획득

스레드 네트워킹, 매장 방문 영업, FCM 푸시 재유입, 앱 내 유도 팝업, GEO·SEO 설정

개발 · 배포

SwiftUI, Android, Next.js, TypeScript, Firebase(Firestore·Cloud Functions·Auth·App Hosting), StoreKit 2·Google Play Billing 영수증 검증, SOLAPI 알림톡

AI

Claude Code 역할별 세션 운영(전략·기획·디자인·개발·QA), Figma MCP·서브에이전트 기반 도구 제작, LLM 판정 기반 업무 자동화

데이터

SQL(Oracle·MySQL·PostgreSQL), Cloud Firestore

자동화 · 운영

일일 지표 리포트 봇, 스토어 출시 감시 Slack 봇, Maestro, GitHub Actions, Jenkins

AWS Certified Solutions Architect – Associate · 2024.02 | OPIc Intermediate High · 2022.09 | 경기대학교 국어국문학과 졸업